PYTHON

Module 23: Sequence Generation & Range

The range() Function

The `range()` function is a built-in Python tool that returns an immutable sequence of numbers. At CodeHive, we use it primarily to control loop execution and generate systematic numerical data.

- **Data Type**: Numbers generated by `range()` belong to their own specific class: `<class 'range'>`.

- **Immutability**: Once a range is defined, its sequence and values cannot be modified.

Syntax & Arguments

The function can be customized using up to three arguments:

- **One Argument**: `range(stop)` - Starts at 0 and ends before 'stop'.
- **Two Arguments**: `range(start, stop)` - Starts at 'start' and ends before 'stop'.
- **Three Arguments**: `range(start, stop, step)` - Adds a 'step' increment (or decrement) to the sequence.

Rule: The 'start' value is inclusive, but the 'stop' value is always exclusive.

Display & Conversion

Because `range` is an object used for iteration, printing it directly (e.g., `print(range(5))`) will simply output the range definition.

- **Visualization**: To view the actual numbers, you must convert it to a collection type like a `list` or `tuple`.

Example: `list(range(5))` produces `[0, 1, 2, 3, 4]`.

Sequence Operations

Despite being a generator-like object, ranges support many sequence operations:

- **Indexing**: Access specific numbers using `r[index]`.
- **Slicing**: Extract sub-ranges using `r[start:stop:step]`.
- **Membership**: Check if a number exists in a range using the `in` operator.
- **Length**: Use `len(r)` to find out how many numbers the range contains.

Common Use Cases

- **Iterating**: Running a `for` loop a known number of times.

- **Index Generation**: Creating indices to iterate through a list while having access to the current position.

- **Batching**: Generating step-based triggers (e.g., every 5th item) using the third argument.